



Kubernetes Networking

Lab 3: Ingress

From Many Services to One Entry Point: Kubernetes Ingress in Action

Lab Objective

By the end of this lab, participants will be able to:

- Explain why Ingress exists in Kubernetes
 - Differentiate between Service, LoadBalancer, and Ingress
 - Understand the role of an Ingress Controller
 - Configure path-based and host-based routing
 - Trace external traffic flow from client → ingress → service → pod
-

Prerequisites

- A running Kubernetes cluster (Minikube, Kind, or managed cluster)
- kubectl configured and working

- Basic understanding of:
 - Pods
 - Deployments
 - Services (ClusterIP)
-

Mental Model

Ingress provides a single HTTP(S) entry point into the cluster.

Instead of exposing every service with its own LoadBalancer or NodePort, Ingress allows:

- One external IP
- Centralized routing rules
- Central TLS termination
- Clean URLs and domains

Think of Ingress as:

| A smart reverse proxy running inside your Kubernetes cluster.

Lab Architecture Overview

External Client

- Ingress Controller (NGINX)
 - Kubernetes Ingress Rules
 - Service (ClusterIP)
 - Pod
-

Step 1 – Deploy Sample Applications

We will deploy two simple applications:

- **web-app** – returns "Web App."

- **api-app** – returns "API App."

Each application consists of:

- Deployment
- ClusterIP Service

Key Teaching Point

At this stage, both services are internal only. They cannot be accessed from outside the cluster.

Step 2 – Install the Ingress Controller

Important Concept

Ingress resources do nothing on their own.

They require an Ingress Controller to:

- Watch Ingress resources
- Configure a real reverse proxy
- Handle incoming traffic

We will use NGINX Ingress Controller.

Validation

Confirm the controller is running:

```
kubectl get pods -n ingress-nginx
```

If the controller is not running, Ingress traffic will not work.

Step 3 – Create an Ingress Resource (Path-Based Routing)

Create an Ingress that routes traffic based on URL paths:

- `/web` → web-service

- `/api` → api-service

Example Ingress Definition

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: demo-ingress
spec:
  rules:
  - http:
      paths:
      - path: /web
        pathType: Prefix
        backend:
          service:
            name: web-service
            port:
              number: 80
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 80
```

Key Teaching Points

- One Ingress can route to **multiple services**
- `pathType: Prefix` ensures subpaths are matched
- Ingress operates at **Layer 7 (HTTP)**

Step 4 – Access the Applications

Retrieve the Ingress IP:

```
kubectl get ingress
```

Test access:

```
curl http://<INGRESS-IP>/web  
curl http://<INGRESS-IP>/api
```

Observation

- Same IP
- Same port (80)
- Different backend services

This is the core value of Ingress.

Step 5 – Host-Based Routing (Optional Extension)

Ingress can also route traffic based on **hostnames**.

Example

```
rules:  
- host: api.demo.local  
  http:  
    paths:  
    - path: /  
      pathType: Prefix  
      backend:  
        service:  
          name: api-service  
- host: web.demo.local  
  http:  
    paths:  
    - path: /
```

```
pathType: Prefix
backend:
  service:
    name: web-service
```

Update your local hosts file:

```
<INGRESS-IP> api.demo.local
<INGRESS-IP> web.demo.local
```

Teaching Point

This mirrors real-world production setups using domains and subdomains.

Step 6 – TLS Termination

Ingress is typically responsible for:

- TLS termination
- Certificate management (via cert-manager)
- Forwarding decrypted HTTP traffic internally

Traffic flow:

Client (HTTPS)

- Ingress (TLS termination)
 - Service (HTTP)
 - Pod
-

Common Mistakes and Debugging

Issue	Explanation
Ingress exists but no traffic	No Ingress Controller
404 from Ingress	Path mismatch
Service works but Ingress fails	Wrong service name or port

Issue	Explanation
Expecting TCP/UDP routing	Ingress is HTTP-focused

Comparison Summary

Component	Responsibility
Service	Internal load balancing
LoadBalancer	External exposure
Ingress	HTTP routing and entry management

Break-It Challenges

1. Delete the Ingress Controller and observe failure
 2. Change the Ingress path and trigger 404 errors
 3. Point Ingress to a non-existing service
 4. Delete backend pods and observe traffic continuity
-

Key Takeaways

- Ingress provides a **single, scalable entry point**
 - Ingress requires a controller to function
 - Services remain internal and simple
 - Ingress enables clean URLs, TLS, and routing logic
-

Outcome

After this lab, participants should confidently explain and demonstrate how Kubernetes handles external HTTP traffic using Ingress.